

Listas Doblemente Ligadas

Introducción

Los arreglos y las estructuras de datos (listas, colas, pilas, etc.) son entes informáticos abstractos que nos permiten almacenar datos, es decir, en un lenguaje de programación como Java, objetos y/o tipos primitivos (int, double, char, boolean, etc...).

Los arreglos son, probablemente, la estructura más usada para almacenar y ordenar datos dentro de la programación, debido a que la sintaxis para acceder a sus datos es muy amigable para el programador. Por ejemplo, si tenemos un arreglo de int's, y queremos sumar el segundo número de un arreglo con el cuarto, simplemente usamos la notación [] para acceder a ellos.

Pero esta forma de almacenamiento tiene ciertas desventajas, por ejemplo:

1. Los arreglos tienen una capacidad de almacenamiento determinada y no modificable, lo que se transforma realmente en un problema si queremos almacenar datos sin saber cuántos almacenaremos en total, o si la cantidad de datos es variable.
2. Por (1), los programadores utilizan arreglos "lo suficientemente grandes", desperdiciando espacio de memoria que nunca se utiliza. Por ejemplo, si queremos registrar a los clientes que compraron en nuestra tienda cada día, podríamos crear un arreglo de una capacidad estimada de 100 espacios para cada día. Pero en el caso que en una ocasión sólo entraran a comprar 22 clientes, 78 espacios del arreglo son desperdiciados, y, por lo tanto, espacio en la memoria es desperdiciado. O peor aún, que entraran 150 personas a comprar y que nuestro arreglo no fuera capaz de almacenar a todos los clientes. En estos casos, los arreglos no son la herramienta indicada.
3. La inserción de un objeto al principio del arreglo, sin sobrescribir el primer espacio, se torna más complicada, pues debemos correr en un espacio a la derecha a todo el resto de los datos. Es decir, los arreglos no manejan los datos de forma dinámica.

Para arreglar este problema, haremos uso de una nueva forma de almacenamientos: las estructuras de datos basadas en punteros, con su principal exponente, **la Lista Enlazada**.

Para entender este tipo de estructuras es necesario tener claro algunos conceptos:

puntero/apuntado

un puntero es un espacio en la memoria que almacena una referencia o dirección de memoria de otra variable, conocida como su apuntado. El valor de un puntero puede ser null, lo que significa actualmente no se refiere a ningún apuntado.

referencia

la operación de referencia en un puntero sirve para tener acceso a su apuntado.

Asignación de puntero

una operación de asignación entre dos punteros, como $p=q$, hace que los dos punteros se refieran (o apunten) al mismo apuntado. No se copia dos veces en la memoria al apuntado, sino que los dos punteros almacenan la dirección de memoria del apuntado.

Ahora bien, para solucionar el problema encontrado al usar arreglos en ese tipo de situaciones, introduciremos un nuevo tipo de estructura de almacenamiento: **Las Estructuras de Datos en Base a Punteros.**

Uso de listas.

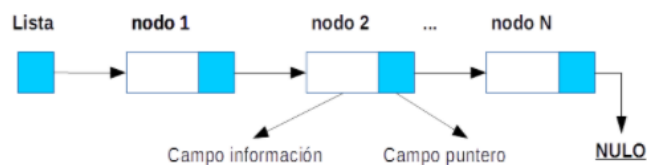
- *Lista Enlazada Simple o Lista Simplemente Enlazada*
- **Listas Doblemente Enlazadas**
- *Listas Circulares*

¿Qué es una lista?

La Lista Enlazada Simple es la más fundamental estructura de datos basada en punteros, y del concepto fundamental de ésta derivan las otras estructuras de datos.

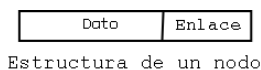
Para solucionar un problema como el presentado anteriormente, necesitamos una estructura que, al contrario de los arreglos, sea capaz de modificar su capacidad, es decir, que maneje los datos de forma dinámica. Para lograr esto, nace la idea de **lista enlazada**.

Lista simplemente enlazada

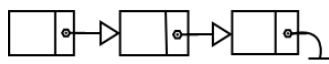


Un arreglo asigna memoria para todos sus elementos ordenados como un sólo bloque. En cambio, la lista enlazada asigna espacio para cada elemento por separado, en su propio bloque de memoria, llamado nodo. La lista conecta estos nodos usando punteros, formando una estructura parecida a la de una cadena.

Un **nodo** es un objeto como cualquier otro, y sus atributos serán los encargados de hacer el trabajo de almacenar y apuntar a otro nodo. Cada nodo tiene dos atributos: un atributo “contenido”, usado para almacenar un objeto; y otro atributo “siguiente”, usado para hacer referencia al siguiente nodo de la lista.

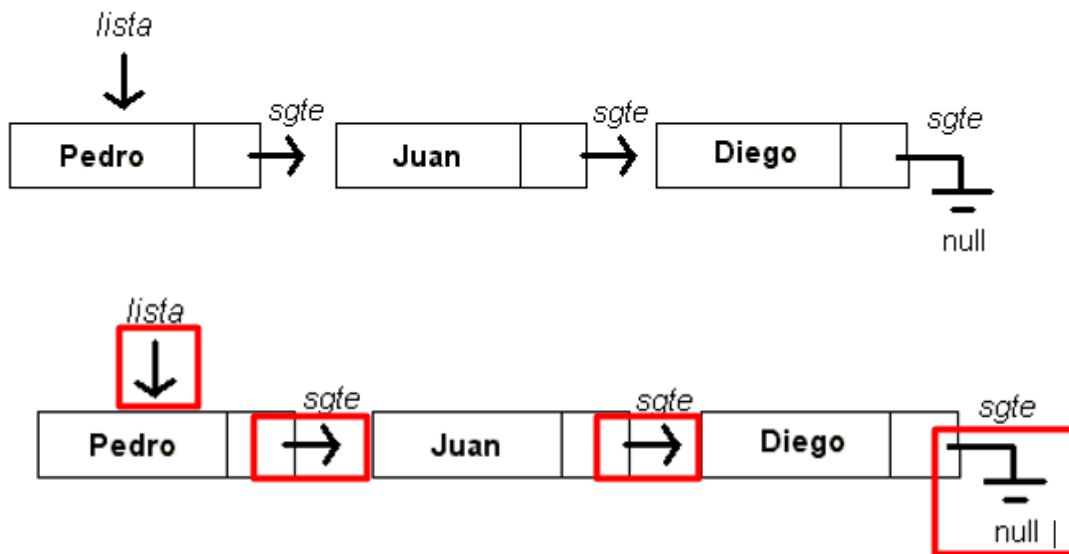


Estructura de un nodo



Lista enlazada

Los nodos serán representados por los siguientes símbolos:



Puntero al siguiente nodo



Nodo cuyo puntero tiene no apuntado (apuntando a null)

La parte frontal de la lista es representada por un puntero al primer nodo. Es decir, un atributo de la lista enlazada es un nodo primero.

Implementación (Definición) de una lista simple:

Para implementar una lista simple enlazada, se define una clase Nodo, la cual indica el tipo de los objetos guardados en los nodos de la lista. Para este caso se asume que los elementos son String (puede ser cualquier tipo). También es posible definir nodos que puedan guardar tipos arbitrarios de elementos.

Dada la clase Nodo, se puede definir una clase, ListaSimple, definiendo la lista enlazada actual. Esta clase guarda una referencia al nodo cabeza y una variable va contando el número total de nodos.

```

1 //Declaraciones generales
2
3 Estructura tipo_nombre
4 -----
5 estructura tipo_nombre siguiente
6
7 Fin de estructura
8
9 tipo_nombre a inic, posic, anterior
10
11 función crear
12     reservar (inic)
13     inicasiiguiente = nulo
14
15     inicaelemento = dato
16
17     posic = inic
18
19 fin función
20

```

Operaciones sobre una Lista

Estos métodos son los más básicos que podemos crear a partir de la "lógica del puntero", y la implementación de otros métodos basados en la misma idea queda a gusto (o necesidad) del programador. Por ejemplo, podríamos crear métodos que filtren la lista según una característica de los clientes (como discriminar entre los clientes frecuentes y los no frecuentes), o un método que traspase los datos de los clientes a un archivo de

texto para guardarlos, y otro que sea capaz de leer este archivo y llenar una lista con los datos guardados. En fin, hay infinitas posibilidades.

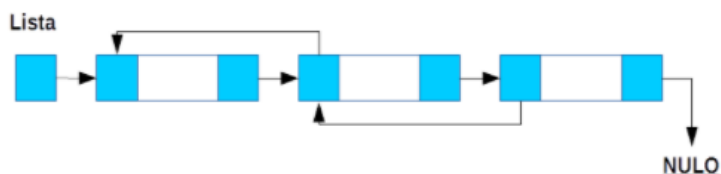
Para una lista L, x el elemento perteneciente a L, p la posición en L, entonces se tiene:

- END(L): Retorna la posición siguiente al último elemento de la lista.
- FIRST(L): Retorna la posición del primer elemento de la lista. Si L está vacía, retorna lo mismo que END(L).
- INSERT (x, p, L): Inserta x en la posición p de L. Si p es END(L), entonces x se agrega al final
- DELETE (p, L): Elimina el elemento de la posición p en L. Si p es END(L) o p no existe, se indefine.
- LOCATE (x, L): Retorna la posición de x en L. Si x esta repetido, retorna la primera vez que aparece. Si x no existe retorna END(L).
- RETRIEVE (p, L): Retorna el elemento en la posición p de L. Si p es END(L) o p no existe, se indefine.
- NEXT (p, L): Retorna la posición siguiente a p en L. Si p es END(L) o p no existe, se indefine.
- PREVIOUS (p, L): Retorna la posición precedente a p en L. Si p es FIRST(L) o p no existe, se indefine.
- MAKENULL(L): Deja L como lista vacía y retorna la posición END(L).
- PRINTLIST(L): Muestra los elementos de L en orden de ocurrencia.

Listas Doblemente Enlazadas

Un tipo de lista enlazada que permite ir en ambas direcciones, hacia adelante y hacia atrás, en una lista enlazada. Esta es la lista doblemente enlazada. Tal lista permite una gran variedad de operaciones rápidas de actualización, incluyendo la inserción y el borrado en ambos extremos, y en el centro. Un nodo en una lista doblemente enlazada guarda dos referencias un enlace sig., el cual apunta al siguiente nodo en la lista, y un enlace previo, el cual apunta al nodo previo en la lista.

Lista doblemente enlazada



El almacenamiento con nexos en sólo un sentido tiene el problema de no permitir recorrido hacia atrás, pero si se agrega un segundo apuntador se puede lograr este objetivo. —Pero tienen el problema de requerir más espacio de memoria.

La ventaja que proporciona esta estructura es la operación de búsqueda e inserción, recorriendo hacia atrás en forma inmediata.

Una implementación de un nodo de una lista doblemente enlazada se muestra en el siguiente código, donde nuevamente se asume que los elementos son

cadenas de caracteres.

¿Cómo se definen?

Implementación (Recordemos):

Para definir un elemento de la lista será utilizado el tipo **struct**.

El elemento de la lista contendrá un campo **dato**, un puntero **anterior** y un puntero **siguiente**.

```
typedef struct dl_ElementoLista {  
    char *dato;  
    struct dl_ElementoLista *anterior;  
    struct dl_ElementoLista *siguiente;  
}dl_Elemento;
```

Operaciones sobre la lista doblemente enlazadas

Inicialización

Modelo de la función:

```
void inicialización (Lista *lista);
```

Esta operación tiene que ser realizada antes de otra operación sobre la lista.

Esta empezará indicando el puntero inicio y también el puntero fin con el puntero NULL y, además, el tamaño con el valor 0.

La función:

```
void inicialización (Liste *liste){  
    lista->inicio = NULL;  
    lista->fin = NULL;  
    tamaño = 0;  
}
```

Inserción de un elemento en la lista

El siguiente es el **algoritmo de inserción** y el registro de los elementos: declaración del elemento que se insertará, asignación de la memoria para el nuevo elemento, completar el contenido del campo de datos, actualizar los punteros hacia el elemento anterior y el elemento siguiente, actualizar los punteros hacia el primero y último elemento si es necesario. **Caso específico:** en una lista con un solo elemento, el primero es al mismo tiempo el último. Actualizar el tamaño de la lista.

Para añadir un elemento a la lista hay **varias situaciones**: insertar en una lista vacía, al inicio de la lista, al final de la lista, antes de un elemento y después de un elemento.

Función:

```
int ins_en_lista_vacia (dl_Lista *lista, char *dato);
```

La función en caso de error da como resultado **-1**, si no devuelve **0**.

Las **etapas** son las siguientes: asignación de memoria para el nuevo elemento, llenado del campo de datos del nuevo elemento, el puntero **anterior** del nuevo elemento apuntará hacia **NULL** (dado que la inserción es hecha en una lista vacía utilizamos la dirección del puntero **inicio** que vale **NULL**), el puntero **siguiente** del nuevo elemento apuntará hacia **NULL** (debido a que la inserción es hecha en una lista vacía se utiliza la dirección del puntero **fin** que vale **NULL**), los punteros de **inicio** y **fin** señalarán hacia el nuevo elemento y el tamaño es actualizado:

La función:

```
int insercion_en_lista_vacia (dl_Lista * lista, char *dato){
    dl_Elemento *nuevo_elemento;
    if ((nuevo_elemento = alloc (nuevo_elemento)) == NULL)
        return -1;
    strcpy (nuevo_elemento->dato, dato);
    nuevo_elemento->anterior = lista->inicio;
    nuevo_elemento->siguiente = lista->fin;
    lista->inicio = nuevo_elemento;
    lista->fin = nuevo_elemento;
    lista->tamaño++;
    return 0;
}
```

Inserción al inicio de la lista

Modelo de la función:

```
int ins_inicio_lista(dl_Lista * lista, char *dato);
```

La función devuelve como resultado **-1** en caso de error. En caso contrario, devuelve **0**.

Las **etapas** son las siguientes: asignación de memoria al nuevo elemento, rellenado del campo de datos del nuevo elemento, el puntero **anterior** del nuevo elemento apunta hacia **NULL**, el puntero **siguiente** del nuevo elemento apuntará hacia el primero de los elementos, el puntero **anterior** del primer elemento apunta hacia el nuevo elemento, el puntero **inicio** apunta hacia el nuevo elemento, el puntero **fin** no cambia y el tamaño es incrementado:

La función devuelve como resultado **-1** en caso de error. En caso contrario, devuelve **0**.

Las **etapas** son las siguientes: asignación de memoria al nuevo elemento, rellenado del campo de datos del nuevo elemento, el puntero **anterior** del nuevo elemento apunta hacia **NULL**, el puntero **siguiente** del nuevo elemento apuntará hacia el primero de los elementos, el puntero **anterior** del primer elemento apunta hacia el nuevo elemento, el puntero **inicio** apunta hacia el nuevo elemento, el puntero **fin** no cambia y el tamaño es incrementado:

La función:

```
int ins_inicio_lista(dl_Lista * lista, char *dato){
    dl_Elemento *nuevo_elemento;
    if ((nuevo_elemento = alloc (nuevo_elemento)) == NULL)
        return -1;
    strcpy (nuevo_elemento->dato, dato);
    nuevo_elemento->anterior = NULL;
    nuevo_elemento->siguiente = lista->inicio;
    lista->inicio->anterior = nuevo_elemento;
    lista->inicio = nuevo_elemento;
    lista->tamaño++;
    return 0;
}
```

Insertar un elemento al final de la lista

Modelo de la función:

```
int ins_fin_lista(dl_Lista * lista, char *dato);
```

La función devuelve el valor de **-1** en caso de error, si no devuelve **0**.

Las **etapas** son las siguientes: asignación de memoria al nuevo elemento, rellenado del campo de datos del nuevo elemento y el puntero **siguiente** del nuevo elemento va a apuntar hacia **NULL**, el puntero **anterior** del nuevo elemento apunta hacia el último elemento (es el puntero **fin** que contiene por ahora su dirección), el puntero **siguiente** del último elemento va a apuntar al nuevo elemento, el puntero **fin** apunta hacia el nuevo elemento, el puntero **inicio** no cambia y el tamaño es incrementado:

La función:

```
int ins_fin_lista(dl_Lista * lista, char *dato){
    dl_Elemento *nuevo_elemento;
    if ((nuevo_elemento = alloc (nuevo_elemento)) == NULL)
        return -1;
    strcpy (nuevo_elemento->dato, dato);
    nuevo_elemento->siguiente = NULL;
    nuevo_elemento->anterior = lista->fin;
    lista->fin->siguiente = nuevo_elemento;
    lista->fin = nuevo_elemento;
    lista->tamaño++;
    return 0;
}
```

Insertar un elemento en una posición determinada

Insersión antes de un elemento de la lista

Modelo de la función:

```
int ins_antes (dl_Lista * lista, char *dato, int pos);
```

```
int ins_antes (dl_Lista * lista, char *dato, int pos);
```

La inserción se efectuará antes de cierta posición pasado como argumento a la función. La posición indicada no debe ser **ni el primer ni el último elemento**. En ese caso hay que utilizar las funciones de inserción al inicio y/o al final de la lista.

Las **etapas** son las siguientes: asignación de memoria al nuevo elemento, rellenado del campo de datos del nuevo elemento, elección de una posición en la lista (la inserción se hará después de la posición elegida), el puntero **siguiente** del nuevo elemento apunta hacia el elemento actual, el puntero **anterior** del nuevo elemento apunta en la dirección hacia la que apunta el puntero **anterior** del elemento actual, el puntero **siguiente** del elemento que precede al elemento actual apuntará hacia el nuevo elemento, el puntero **anterior** del elemento actual apunta hacia el nuevo elemento, el puntero **fin** no cambia, el puntero **inicio** cambia si la posición elegida es la posición **1** y el tamaño se incrementa en una unidad:

La función:

```
int ins_antes (dl_Lista * lista, char *dato, int pos){
```



```

int i;
dl_Elemento *nuevo_elemento, *actual;
if ((nuevo_elemento = alloc (nuevo_elemento)) == NULL)
    return -1;
strcpy (nuevo_elemento->dato, dato);
actual = lista->inicio;
for (i = 1; i < pos; ++i)
    actual = actual->siguiente;
nuevo_elemento->siguiente = actual;
nuevo_elemento->anterior = actual->anterior;
if(actual->anterior == NULL)
    lista->inicio = nuevo_elemento;
else
    actual->anterior->siguiente = nuevo_elemento;
actual->anterior = nuevo_elemento;
lista->tamaño++;
return 0;
}

```

Inserción después de un elemento de la lista

Modelo de la función:

```
int ins_después (dl_Lista * lista, char *dato, int pos);
```

La función devuelve **-1** en caso de error. De lo contrario devuelve **0**. La inserción se efectuará después de que cierta posición haya pasado como argumento a la función. La posición indicada no debe ser el último elemento. En ese caso hay que utilizar la función de inserción al final de la lista.

Las **etapas** son las siguientes: asignación de memoria al nuevo elemento, rellenado del campo de datos del nuevo elemento, elección de una posición en la lista (la inserción se hará después de la posición elegida), el puntero **siguiente** del nuevo elemento apunta en la dirección del puntero **siguiente** del elemento actual (ver la imagen), el puntero **anterior** del nuevo elemento apunta hacia el elemento actual, el puntero **anterior** del elemento que va después del elemento actual apuntará hacia el nuevo elemento, el puntero **siguiente** del elemento actual apunta hacia el nuevo elemento, el puntero **inicio** no cambia, el puntero **fin** cambia si la posición elegida es la posición del último elemento (el tamaño de la lista) y el tamaño se incrementa en una unidad:

La función:

```
int ins_después (dl_Lista * lista, char *dato, int pos){
    int i;
    dl_Elemento *nuevo_elemento, *actual;
    if ((nuevo_elemento = alloc (nuevo_elemento)) == NULL)
        return -1;
    strcpy (nuevo_elemento->dato, dato);
    actual = lista->inicio;
    for (i = 1; i < pos; ++i)
        actual = actual->siguiente;
    nuevo_elemento->siguiente = actual->siguiente;
    nuevo_elemento->anterior = actual;
    if(actual->siguiente == NULL)
        lista->fin = nuevo_elemento;
    else
        actual->siguiente->anterior = nuevo_elemento;
    actual->siguiente = nuevo_elemento;
    lista->tamaño++;
    return 0;
}
```

Eliminar un elemento

Eliminación de un elemento de la lista

A continuación, el algoritmo para **eliminar un elemento de la lista**: utilización de un puntero temporal para almacenar la dirección de los elementos a eliminar, el elemento a eliminar puede encontrarse en cualquier posición en la lista.

En relación a la [lista enlazada simple](#) en el que la eliminación sólo puede ser hecha después que un elemento ha sido designado, las listas doblemente enlazadas **son más flexibles** gracias a los 2 punteros que permiten guardar el rastro tanto hacia atrás como hacia delante.

Luego hay que **liberar la memoria** ocupada por el elemento eliminado y actualizar el tamaño de la lista.

Para eliminar un elemento de la lista existen **varias situaciones**: eliminación al inicio de la lista, eliminación al final de la lista, eliminación antes de un elemento, eliminación después de un elemento.

Eliminación de un elemento

Sin embargo, la eliminación al inicio y al final de la lista doblemente enlazada, así como antes o después de un elemento equivale a la **eliminación en la posición 0** (cero) o en la **posición N** (**N** = número de elementos de la lista) o en otra parte de la lista.

En el caso de listas doblemente enlazadas la eliminación en cualquier posición no presenta ningún problema gracias a los punteros **anterior** y **siguiente**, que permiten conservar el enlace entre los elementos de la lista. Razón por la cual solo vamos a crear una función. Si deseamos eliminar el elemento al inicio de la lista elegiremos la posición cero, Si deseamos eliminar el elemento al final de la lista elegiremos la posición **N** (el número de elementos), si deseamos eliminar cualquier elemento entonces elegimos su posición en la lista.

Eliminación en la lista

Modelo de la función:

```
int supp(dl_Lista *lista, int pos);
```

La posición elegida es igual al número de elementos de la lista, el puntero **sup_elemento** contendrá la dirección del último elemento, hacemos apuntar al puntero **siguiente** del penúltimo elemento (es el elemento hacia el que apunta el puntero **anterior** del último elemento), hacia **NULL** y actualizamos el puntero **fin**:

La posición elegida es **aleatoria** en la lista, el puntero **sup_elemento** contendrá la dirección del elemento a eliminar, el puntero **siguiente** del elemento que antecede al elemento a eliminar apunta hacia la dirección contenida en el puntero **siguiente** del elemento a eliminar, el puntero **anterior** del elemento que va después del elemento a eliminar apunta hacia la dirección contenida en el puntero **anterior** del elemento a eliminar y el tamaño de la lista será disminuida en 1 elemento:

```

int supp(dl_Lista *lista, int pos){
    int i;
    dl_Elemento *sup_elemento,*actual;
    if(lista->tamaño == 0)
        return -1;

    if(pos == 1){ /* eliminación del 1er elemento */
        sup_elemento = lista->inicio;
        lista->inicio = lista->inicio->siguiente;
        if(lista->inicio == NULL)
            lista->fin = NULL;
        else
            lista->inicio->anterior == NULL;

    }else if(pos == lista->tamaño){ /* eliminación del último elemento */
        sup_elemento = lista->fin;
        lista->fin->anterior->siguiente = NULL;
        lista->fin = lista->fin->anterior;
    }else { /* eliminación en otra parte */
        actual = lista->inicio;
        for(i=1;i<pos;++i)
            actual = actual->siguiente;
        sup_elemento = actual;
        actual->anterior->siguiente = actual->siguiente;
        actual->siguiente->anterior = actual->anterior;
    }
    free(sup_elemento->dato);
    free(sup_elemento);
    lista->tamaño--;
    return 0;
}

```

BIBLIOGRAFÍA DE LA INVESTIGACIÓN

http://agrega.juntadeandalucia.es/repositorio/02122016/a5/es-an_2016120212_9131705/33_listas.html

<https://www.programacion.com.py/escritorio/c/listas-en-c>

<https://sites.google.com/site/edatosubo/>

<https://es.ccm.net/faq/2872-listas-doblemente-enlazadas#insercion-de-un-elemento-en-la-lista>